

Guía maestra: diseño en Inventor y animaciones con IA (método DEFYMOTION / InventorIA)

Versión 1.0 · 7 de octubre de 2026

Basada en todo el trabajo real del proyecto InventorIA:

- grippers paramétricos;
- rejas;
- planos A4 y DXF;
- listas de materiales;
- simulaciones de celdas KUKA (ADECO, CELDA_KR140, NewPack2).

Para qué sirve: arrancar un proyecto nuevo desde cero (diseño o animación) con una IA que maneja Autodesk Inventor por script, sin repetir los errores que ya costaron horas.

Cómo usarla:

1. Pasale este archivo completo a la IA al empezar el proyecto (sección 0).
2. Usá las listas de control de la sección 15.
3. Copiá las plantillas de código de la sección 16.

Índice

0. [Arranque de un proyecto nuevo \(texto para pegarle a la IA\)](#)
1. [Reglas de oro](#)
2. [Infraestructura: PC, Python y vigilante](#)
3. [Conexión con Inventor y utilidades básicas](#)
4. [Matrices, unidades y sistemas de coordenadas](#)
5. [Relevar antes de diseñar](#)
6. [Piezas](#)
7. [Ensamblajes](#)
8. [Diseño de grippers \(reglas y cálculo\)](#)
9. [Planos de fabricación y DXF](#)
10. [Listas de materiales \(Excel\)](#)
11. [Animaciones: arquitectura del simulador](#)
12. [Animaciones: cómo se mueve cada cosa](#)
13. [Animaciones: cámara, luz, calidad y video](#)
14. [Animaciones: proceso, diagnóstico y retoques sin rehacer](#)

- 15. [Listas de control](#)
- 16. [Plantillas de código](#)
- 17. [Errores típicos y su solución](#)
- 18. [Datos de referencia](#)

0. Arranque de un proyecto nuevo

Texto para pegarle a la IA al empezar (completar lo que está entre < >):

```
None
Vas a trabajar con Autodesk Inventor 2025 en mi PC (Windows) siguiendo la "Guía
maestra" adjunta.
Empresa: <DEFYMOTION>, idioma: español rioplatense, unidades mm, hojas A4.
Proyecto: <nombre corto, sin espacios> → carpeta
Documents\InventorIA\Generados\<NOMBRE>\
Tipo: <diseño de gripper | reja | estructura | simulación de celda | otro>
Originales (NO tocar, sólo leer): < rutas >
Datos de entrada: <caja LxAxH y peso, cantidad, robot, layout del cliente, patrón
de palletizado, etc.>
Entregables: <ensamble con restricciones, planos A4 PDF/DWG, DXF, BOM xlsx, video
Full HD horizontal/vertical...>
El vigilante está <abierto/cerrado>. Último pedido usado: <pNNN>.
Antes de construir: relevá lo existente y mostrame una imagen de prueba.
```

Preguntas que la IA tiene que hacer **antes** de un trabajo caro, si el pedido no las contesta:

Tema	Pregunta
Caja / producto	¿Medidas exactas (L × A × H) y peso? ¿Qué lado aprietan las palas?
Patrón	¿Imagen o medidas del patrón por piso? ¿Cómo rota entre pisos? ¿Cuántos pisos?
Robot	¿Modelo exacto (por ejemplo KR 30 R2100 vs KR 30-3)? ¿Patrón de la brida?
Ciclo	¿Cada cuánto llega una caja? ¿Toma pallet y cartón también?
Layout	¿Qué ensamble es el del cliente? ¿Está abierto en Inventor?

Tema	Pregunta
Video	¿Horizontal o vertical? ¿Qué zooms quiere ver?
Ambigüedades del mecanismo	Mostrar una foto de prueba ("¿así cierran las garras?") antes de renderizar todo

1. Reglas de oro

1. Nunca tocar originales.

- Los proyectos reales (`Documents\De1 C\DEFYOTION\...` o lo que el usuario indique) se abren **sólo para leer**.
- Se trabaja sobre **copias** (`SaveAs`) o sobre archivos nuevos en `Documents\InventorIA\Generados\<NOMBRE>\`.
- Si una prueba puede mover algo, por ejemplo cambiar restricciones para medir: **sólo en copias**. Ya pasó que una prueba de perturbación corrió las garras en el modelo abierto del usuario.

2. Editar, no regenerar. Si se pide corregir algo ya hecho:

- plano → editar el dibujo existente; borrar y rehacer sólo la hoja mala;
- pieza o ensamble → modificar el existente (parámetros, posiciones; `--reusar-piezas, --rehacer <pieza>`);
- video → re-renderizar **sólo el tramo** que cambia y empalmarlo (sección 14.4).

3. Todo por script, con argumentos.

- Cada tarea es un `.py` reutilizable que escribe un `resultado.json`.
- La segunda vez se corre el mismo script con otros parámetros.

4. Medir antes de suponer.

- Releva cajas envolventes, matrices, restricciones y caras del modelo real, y guardarlo en JSON.
- Las suposiciones ("el pallet mide 1000 en X", "el origen está en el centro") se verifican con `RangeBox`.

5. Verificar después de construir, y mirar.

- Comprobar interferencias, envolvente, masa y la imagen ISO.
- **No decir que algo funciona sin haber mirado la imagen, el PDF o el cuadro del video.**

6. No guardar las pruebas.

- Animaciones y pruebas mueven el modelo en memoria y al final restituyen todo (bloque `finally`).
- `Save()` sólo en archivos propios recién generados o en copias de escena.

7. Probar en chico antes de lo caro:

1. una imagen de prueba (encuadre, mecanismo);
 2. 1 cuadro;
 3. un tramo corto;
 4. recién después, el render completo.
2. **Convenciones DEFYMOTION:**
- mm y hojas A4, todas las hojas en un mismo archivo de dibujo;
 - sin cotas de radio de plegado;
 - pala fija sin hexágonos;
 - videos Full HD con "Luz suave", sombras y planos de trabajo ocultos.
2. **Comunicar en corto:**
- al empezar, una frase con lo que se va a hacer y cuánto tarda;
 - al terminar, qué salió, dónde está en la PC y qué falta.
3. **Seguridad de los cálculos.**
- Los cálculos de retención, estructura, neumática y carga del robot son preliminares.
 - Los aprueba un ingeniero responsable antes de fabricar.

2. Infraestructura

2.1 Qué hay que tener en la PC

Elemento	Detalle
Autodesk Inventor 2025	Con licencia, abierto normalmente por el usuario
Python 3.13 (real, no el alias de la Microsoft Store)	<code>setup.ps1</code> lo detecta
Carpeta del complemento	<code>Documents\InventorIA\inventor-assistant-portable\</code> con <code>server\</code> , <code>scripts\</code> , <code>.venv\</code>
Dependencias del <code>.venv</code>	<code>pywin32</code> (312), <code>mcp</code> (opcional), <code>openpyxl</code> (opcional en la PC)
Carpeta de trabajo	<code>Documents\InventorIA\</code> con <code>_pedidos\</code> , <code>Generados\</code> , <code>Analisis\</code> , <code>Comerciales\</code> , copias de proyectos

Elemento	Detalle
Memoria del proyecto	<code>InventorIA_contexto.md</code> , <code>Receta_IA_Inventor.md</code> y esta guía, con copia en <code>inventor-assistant-portable\docs\</code>

Estructura recomendada:

```
None
Documents\InventorIA\
├─ inventor-assistant-portable\
│   ├── server\inventor_mcp.py          # librería de funciones de Inventor (COM)
│   ├── server\gripper_design.py       # diseño paramétrico de grippers
│   ├── server\gripper_calc.py         # cálculo de apriete, cilindros
│   ├── scripts\vigilante.py / .bat     # ejecutor de pedidos
│   ├── scripts\*.py                   # un script por tarea
│   ├── docs\                           # copias de los .md de contexto
│   └─ .venv\                           # se reconstruye con scripts\setup.ps1
├─ _pedidos\                           # cola del vigilante
├─ Generados\<PROYECTO>\               # todo lo que produce la IA
├─ Analisis\<PROYECTO>\               # relevamientos (json + png)
├─ Comerciales\                       # cilindros, guías, etc. copiados para
reusar
└─ <copias de proyectos del cliente>  # p. ej. 12_ADEC02, Simulacion NewPack2
```

2.2 Conexión COM que funciona

- Inventor y el proceso Python tienen que correr **en la misma sesión de Windows** y con el **mismo nivel de permisos**.
- Errores típicos si no:
 - `MK_E_UNAVAILABLE`;
 - `0x80080005` Error en la ejecución de servidor.
- **No usar `gen_py`** (la caché de tipos se rompe). Usar `IDispatch` dinámico (sección 3).
- Si Inventor no está abierto, se puede lanzar `Inventor.exe /Automation`. La ruta se saca del registro: `HKCR\Inventor.Application\CLSID` → `CLSID\{..}\LocalServer32`.

2.3 Cuando la IA no está en la PC: el vigilante (cola de pedidos)

La IA corre en la nube (Linux) y la PC del usuario es Windows. El puente de archivos permite:

- **copiar** archivos a la PC (`device_commit_files`);
- **traerlos** (`device_stage_files`);

- **listar** carpetas.

El vigilante es un Python que corre en la PC y ejecuta los scripts que la IA deja en una carpeta:

None

```

IA (nube)                                PC (Windows + Inventor)
1. escribe scripts\tarea.py — copia —>
inventor-assistant-portable\scripts\tarea.py
2. verifica: la trae de vuelta y compara (diff ignorando \r)
3. escribe _pedidos\pNNN.json — copia —> vigilante lo ve y ejecuta:
.venv\python scripts\tarea.py args...
4. espera                                escribe pNNN.resultado.json y
renombra pNNN.json → pNNN.hecho
5. trae pNNN.resultado.json ← trae —
6. trae imágenes / zip / pdf ← trae — Generados\...\

```

Formato del pedido pNNN.json:

JSON

```

{"script": "simular_newpack.py", "args": ["--velocidad", "1.5",
"--solo-centrador"], "timeout_s": 3600}

```

Opcional: `"instancia": "usuario"`. Sirve si el vigilante corre en modo Inventor privado; el modo por defecto es el compartido.

Resultado pNNN.resultado.json: `id`, `script`, `args`, `segundos`, `codigo_salida`, `salida` (stdout, últimos 60.000 caracteres) y `errores` (stderr).

Reglas del vigilante (todas aprendidas a golpes):

- **Ids siempre nuevos** (p001, p002, ...). Llevar la cuenta y anotarla en el contexto del proyecto.
- **Cualquier *.json de _pedidos se ejecuta como pedido.**
 - Los datos auxiliares van como `.txt` (p048_planta.txt) y se pasan por argumento.
- **Primero el script, después el pedido.** Si el pedido llega antes, corre la versión vieja.
- **El puente puede cachear.** Después de copiar el script:
 1. traerlo de vuelta;
 2. compararlo con `diff <(tr -d '\r' < traído) local`;
 3. si difiere, copiarlo desde una **carpeta de staging nueva** (p. ej. `outputs/p124/`) y verificar otra vez.

- Para **cancelar** un pedido ya copiado, sobrescribir su `.json` con algo inocuo, por ejemplo `{"script":"buscar_archivos.py","args":["zz_no_existe_zz"]}`.
- **Latido:** `vigilante.activo` se actualiza **entre** pedidos. Si no cambia durante un render largo, no es una caída: esperar el `resultado.json`.
- **Si no responde:** casi siempre hay un diálogo de Inventor abierto. Pedir al usuario que lo cierre y reabra `vigilante.bat`.
- Al renombrar a `.hecho` usar `Path.replace(): rename()` falla si ya existe un `.hecho` con ese nombre.
- Sólo ejecuta `.py` de su carpeta `scripts\` (seguridad), con prioridad "debajo de lo normal".
- **Modo compartido (por defecto):** usa el Inventor del usuario. El usuario puede seguir usando la PC y su Inventor, pero conviene que no toque el documento que se está simulando.
- **Inventor privado (`--privado`):** se probó una segunda instancia oculta y **no funcionó de forma confiable**. Quedó opcional.
- **Límites del puente:**
 - 400 MB por archivo y 500 MB por llamada;
 - cuadros de video: zip **sin compresión** (`ZIP_STORED`) y, si pasan 350 MB, partarlos con `partir_zip.py`.
- **Si la PC "no está disponible"** (computadora dormida o app cerrada), pedir al usuario que la active. No repetir acciones que pudieron haberse hecho.

Plantilla mínima del vigilante: sección 16.1.

2.4 Cuando la IA está en la PC

Si la IA tiene una terminal en la misma sesión de Windows que Inventor, se puede saltar el vigilante y correr `.venv\Scripts\python.exe scripts\tarea.py ...` directo. Igual conviene que cada script deje su `resultado.json`.

3. Conexión con Inventor

```
Python
import pythoncom
from win32com.client.dynamic import Dispatch

def inventor(start_if_needed=False):
    pythoncom.CoInitialize()
    unk = pythoncom.GetObject("Inventor.Application")
    return Dispatch(unk.QueryInterface(pythoncom.IID_IDispatch))
```

```

def safe(fn, default=None):          # S(lambda: expr, default): un com_error no
corta el script
    try:
        return fn()
    except Exception:
        return default

def items(col):                      # colecciones COM van de 1..Count
    for i in range(1, col.Count + 1):
        yield col.Item(i)

app = inventor()
app.SilentOperation = True          # SIN diálogos (si no, se cuelga esperando
un clic)
tg = app.TransientGeometry
# ... trabajo ...
app.SilentOperation = False         # siempre restituir (atexit o finally)

```

Reglas de la API:

Regla	Detalle
Unidades internas	cm y radianes. <code>RangeBox</code> , <code>Matrix.Cell(r,4)</code> y <code>Offset.Value</code> vienen en cm: ×10 para mm
Abrir	<code>app.Documents.Open(ruta, visible)</code> . Antes, buscar si ya está abierto comparando <code>FullName</code>
Visible	Invisible para leer; visible si hace falta vista o cámara (imágenes)
Cerrar sin guardar	<code>doc.Close(True)</code> . Después <code>app.Documents.CloseAll(True)</code> cierra sólo los no referenciados
Estados de restricción	Leer <code>sketch.ConstraintStatus</code> después de <code>doc.Update()</code>
Piezas	No tienen <code>Constraints</code> : preguntar <code>DocumentType</code> antes

Regla	Detalle
<code>hasattr</code>	Sobre COM puede tirar <code>com_error</code> : usar <code>safe(lambda: obj.Prop)</code>
Miembros desconocidos	Introspección de la typelib (ver abajo)
Proyecto (<code>.ipj</code>)	Para copias completas de un proyecto, activar su <code>.ipj</code> con espacio de trabajo relativo <code>.</code> y restituir al terminar

Introspección de la typelib:

Python

```
ti = obj._oleobj_.GetTypeInfo(); ta = ti.GetTypeAttr()
nombres = sorted({ti.GetNames(ti.GetFuncDesc(i).memid)[0] for i in
range(ta.cFuncs)})
```

Enumeraciones usadas (Inventor 2025):

- documentos: pieza 12290, ensamble 12291, dibujo 12292, presentación 12293;
- extrusión: unión 20481, corte 20482, sentido positivo 20993, negativo 20994;
- vistas: frente 10764, superior 10754, derecha 10755, ISO 10759;
- estilos de vista: HLR 32258, "desde base" 32260;
- cotas: horizontal 19201, vertical 19202, alineada 19203;
- estado de restricción: completo 51713, sub 51714, sobre 51715;
- hoja A4: `Size = 9997`, apaisada `Orientation = 10242`.

4. Matrices, unidades y coordenadas

Todo el posicionamiento se hace con **listas Python 4x4 en mm**. Se convierte a `Matrix` de Inventor sólo al final:

Python

```
def mread(m): # Matrix de Inventor -> 4x4 en mm
    return [[m.Cell(r, c) * (10 if (c == 4 and r < 4) else 1) for c in range(1,
5)] for r in range(1, 5)]

def mmake(M): # 4x4 en mm -> Matrix de Inventor
```

```

m = tg.CreateMatrix()
m.SetCoordinateSystem(tg.CreatePoint(M[0][3]/10, M[1][3]/10, M[2][3]/10),
                      tg.CreateVector(M[0][0], M[1][0], M[2][0]),
                      tg.CreateVector(M[0][1], M[1][1], M[2][1]),
                      tg.CreateVector(M[0][2], M[1][2], M[2][2]))

return m

def mmul(P, Q): return [[sum(P[i][k]*Q[k][j] for k in range(4)) for j in range(4)]
for i in range(4)]
def inv_rt(M): # inversa de rotación + traslación
    R = [[M[j][i] for j in range(3)] for i in range(3)]
    t = [-sum(R[i][k]*M[k][3] for k in range(3)) for i in range(3)]
    return [R[0]+[t[0]], R[1]+[t[1]], R[2]+[t[2]], [0,0,0,1]]
def tr(x, y, z): return [[1,0,0,x],[0,1,0,y],[0,0,1,z],[0,0,0,1]]
def apply(M, p): return [sum(M[i][k]*(list(p)+[1])[k] for k in range(4)) for i in
range(3)]
def rotv(M, v): return [sum(M[i][k]*v[k] for k in range(3)) for i in range(3)]

```

- **Composición:** $\text{mundo} = T_{\text{padre}} \cdot T_{\text{local}}$.
 - Un componente en un subensamble: $M_{\text{mundo}} = M_{\text{sub_en_top}} \cdot M_{\text{comp_en_sub}}$.
 - Una dirección del mundo (por ejemplo +Y) expresada en las coordenadas de un subensamble: $\text{rotv}(\text{inv_rt}(M_{\text{sub}}), (0, 1, 0))$, que es la fila 2 de la rotación.
- **Mover algo X mm en la dirección del mundo u** dentro de su documento: $M_{\text{nueva}} = \text{tr}(u_{\text{local}} \cdot d) \cdot M_0$, con $u_{\text{local}} = u$ llevado a las coordenadas de ese documento.
- **Y arriba vs Z arriba:**
 - las piezas de Inventor suelen tener Y arriba;
 - los layouts de cliente (ADECO, NewPack2) usan **Z arriba**;
 - el robot KUKA tiene Y arriba en su ensamble;
 - **nunca suponer:** leer las matrices reales.
- **Marco canónico de un objeto** (caja, bandeja, paquete): origen en el centro de la base, X = un lado, Y arriba, Z = el otro lado. Se calcula desde el **RangeBox** del documento de la pieza.
 - Así no importa dónde puso el origen el que la modeló, ni si en el modelo está dada vuelta.
 - $M_{\text{pieza}} = C_{\text{mundo}} \cdot \text{inv}(C_{\text{canónico_en_pieza}})$.
- **Ubicar algo "por lo que mide" y no "por lo que supongo":** antes de un movimiento que debe terminar en contacto (empujar contra un tope), leer el **RangeBox** real y calcular el recorrido para que la cara llegue exacto. Así se absorbe el error de la cinemática inversa (sección 12.4).

5. Relevar antes de diseñar

Todo proyecto nuevo sobre modelos existentes empieza con un **relevamiento sólo lectura** que queda guardado:

Script tipo	Qué vuelca
<code>relevar_activo.py --nombre X</code>	Ensamble activo: árbol a 2 niveles (archivo, matriz, caja, visible, fijo, adaptivo), restricciones con valor y estado, uniones, parámetros, vistas de diseño, representaciones posicionales, 3 imágenes (ISO, planta, frente). Restituye la cámara del usuario
<code>relevar_<proyecto>.py</code>	Componentes en coordenadas del mundo, matrices de los eslabones del robot, comparación con un robot de referencia
<code>inspect_assembly.py</code>	Cantidades, masas, materiales, parámetros, capturas
<code>diag_<cosa>.py</code>	Cajas de las hojas de un subensamble, restricciones de un subdocumento, pruebas puntuales
<code>analizar_brida_kr.py</code>	Patrón de agujeros de la brida de un robot (círculos coplanares en un PCD)
<code>vistas_ensamble.py "<iam>" --arriba 0 0 -1 --dirs="x,y,z;..."</code>	Imágenes con cámara libre
<code>buscar_archivos.py <texto></code>	Dónde está un modelo en la PC

Prueba de perturbación (para entender un mecanismo sin uniones):

1. Recorrer todas las restricciones con offset o ángulo.
2. Cambiar cada una +5 mm o +5°.
3. Medir qué piezas se mueven y cuánto (comparando las cajas).
4. Volver al valor original.

Así se encuentra qué restricción abre una pala o gira una garra. ⚠ **Hacerlo sólo en una copia:** la restitución no siempre deja todo igual.

Qué buscar siempre:

- dónde están los **topes**, los **centros** y las **caras de apoyo** reales (por ejemplo el centrador: cadenas en $z = 360$, tope del fondo $x = -566,4$, tope lateral $y = -1121,2$);
 - **dónde está el origen** de cada pieza que se va a mover. Comparar el `RangeBox` con la traslación de la matriz;
 - qué está **fijo** (`Grounded`), qué está **suprimido** y qué subensambles son **flexibles** (`occ.Flexible`). Esto es clave para animar (sección 12.2);
 - **materiales:** si figura "Genérico", la masa no sirve.
-

6. Piezas

6.1 Chapas (sheet metal) desde el desarrollo —

`build_flat_part(app, spec, ipt, dxf)`

- La pieza se describe en Python como `FlatPart`:
 - contorno exterior (polilínea con *bulges* para arcos);
 - contornos interiores (calados, hexágonos, logo);
 - círculos, ranuras;
 - líneas de plegado con ángulo y radio;
 - espesor y material.
- Se dibuja el desarrollo, se extruye el espesor y se pliega sobre las líneas.
- DXF de corte: desarrollo 1:1, capas `CORTE_EXTERIOR` / `CORTE_INTERIOR`, sin líneas de plegado.
- **Las placas planas también tienen que ser chapa** (convertidas), con el espesor real y el desarrollo sobre la **cara grande**.
 - Si no, el DXF sale del canto. El control automático avisa "CONVERSIÓN A CHAPA INCORRECTA".
- **Logos calados:** no usar `TextBox` (las letras tienen islas que se caen). Usar glifos esténcil propios (polígonos con puentes).
- **Hexágonos de alivianado:** `hex_flats` por pieza (distancia entre caras); van sólo en palas móviles y, en la versión liviana, en el alma del frame y la brida. **La pala fija va sin hexágonos.**
- **Masa estimada antes de construir:** $(\text{área del desarrollo} - \text{agujeros} - \text{hexágonos}) \times \text{espesor} \times 7,85 \text{ g/cm}^3$.

6.2 Piezas paramétricas simples

- Boceto centrado en el origen con `AddAsTwoPointRectangle` (ya trae las restricciones horizontal, vertical y coincidente) y una diagonal de construcción con el punto medio en el origen.
- Cotas visibles con nombre: `AddTwoPointDistance(p1, p2, orientación, punto_texto)`. `AddTwoLineDistance` no existe.
- Extrusión con un parámetro con nombre (`Altura`). Verificar con `RangeBox` y comprobar que el boceto quedó **totalmente restringido** (51713).

6.3 Caños y estructuras

- `build_tube_part`: caño $a \times b \times e$, sección centrada, eje +Z.
- `build_frame_part`: marco soldado (montantes, largueros, travesaños, agujeros).
- **Después de cada extrusión, verificar el sentido** con `RangeBox`. Si salió al revés, borrar la operación y extruir al otro lado.

6.4 iProperties

- `Part Number` de 18 caracteres como máximo (entra en el cajetín).
- `Description`.
- Material real ("Acero, suave"...). Si no, la masa es falsa.

6.5 Comerciales

- **No se modelan.** Se copian de proyectos existentes a `InventorIA\Comerciales\<modelo>` y se insertan desde ahí. Ejemplos: Festo ADN, DSBC; HIWIN HGH/HGW; tuercas DIN.
 - Para enviar un ensamble, dejarlo **autocontenido** con `empaquetar_ensamble.py`:
 1. copia lo externo a `<carpeta>\Comerciales`;
 2. repunta las referencias y guarda;
 3. reabre desde el disco para verificar;
 4. arma el `.zip`.
 - Si un comercial se usó para animar, puede haber quedado "movido" (una pala corrida 4,4 mm). Normalizarlo midiendo una referencia fija (los agujeros de los bloques) antes de copiar posiciones.
-

7. Ensamblajes

7.1 Ubicar componentes

- Calcular la matriz en Python y `Occurrences.Add(ruta, mmake(M))`.
- Mover: `occ.Transformation = mmake(M)`.
- `Grounded = True` en lo que no se mueve.
- Un ensamblaje nuevo puede referenciar archivos existentes sin copiarlos (celda = robot + base + gripper).

7.2 Restricciones

- Inserción en agujeros coaxiales en contacto; coincidencia y nivelación de planos.
- El **signo** de las restricciones de plano no es confiable. **Calibrar**:
 1. crear con offset 0;
 2. medir;
 3. probar candidatos;
 4. quedarse con el que deja la pieza en la pose planificada.
- Restricción de mando con nombre (`Apertura`) para el mecanismo. Probarla moviéndola $\pm$ carrera y midiendo qué se mueve.

7.3 Copias de ensamblajes grandes (orden correcto de `SaveAs`)

Para trabajar sobre un layout del cliente sin tocarlo:

Python

```
doc = app.Documents.Open(LAYOUT, True) # original, NO se guarda
gdoc = occ_gripper.Definition.Document
gdoc.SaveAs(GRIPPER_COPIA, False) # 1º los SUBensambles que voy a modificar
doc.SaveAs(ESCENA_COPIA, False) # 2º el general: ahora referencia a la copia del gripper
```

- Desde ese momento los documentos en memoria **son las copias**: se modifican y se guardan sin riesgo.
- Si el original está abierto en Inventor, pedir al usuario que lo cierre sin guardar antes de copiar.
- Para un sub-subensamblaje (por ejemplo el centrador y, adentro, el tope de pistones y, adentro, el cilindro), repetir el `SaveAs` de **cada nivel** que haya que tocar, del más interno al más externo. Así la cadena de copias queda referenciada.

7.4 Liberar lo que se va a mover

- Suprimir las restricciones que atan las piezas móviles (`c.Suppessed = True`) y sacarles `Grounded`.
- Guardar en un `escena.json`:
 - la lista de suprimidas;
 - la matriz original de cada pieza libre (es la pose "0" de la animación).
- Al robot armado del layout no se le tocan las restricciones con su base: se oculta y se usan sus eslabones sueltos (sección 12.3).

7.5 ⚠ Subensambles **flexibles**

- Si una ocurrencia de subensamblable es **flexible** (`occ.Flexible == True`), la posición de sus componentes **no sale de su .iam**: cada ocurrencia guarda la suya.
- Por eso mover un componente dentro del documento del subensamblable **no se ve**. Fue el caso de la chapa de los pistones del centrador.
- **Cómo moverlo**: usar el **proxy** del componente en el contexto del padre:

Python

```
tp_c = next(o for o in cdoc.ComponentDefinition.Occurrences if o.Name ==
"Tope_pistones_sim:1") # en el padre
chapa = next(o for o in tp_c.SubOccurrences if o.Name == "Chapa tope piston:1")
# proxy
M0 = mread(chapa.Transformation) # en coordenadas del PADRE (cdoc)
chapa.Transformation = mmake(mmul(tr(*[u[i]*d for i in range(3)]), M0)) # u =
dirección en coords del padre
cdoc.Update()
```

- **No apagar Flexible**: al apagarlo, Inventor recalcula el subensamblable y se corren o giran otras piezas (hasta un tope vecino se torció).
 - Los subensambles **rígidos** anidados adentro de uno flexible (por ejemplo un cilindro) sí siguen a su documento: ahí se mueve el componente en su `.iam` y se actualiza ese documento.
-

8. Diseño de grippers

8.1 Reglas de la empresa (DEFYMOTION)

- Toma por **apriete lateral**:
 - **una pala fija** (corrida, sin hexágonos, 3/16" si la caja es alta);
 - **una pala móvil por caja** (adaptada a la caja, sin morder la vecina).
- **Módulo de pala móvil**: pala + cilindro neumático Festo ADN + 2 guías lineales HIWIN + empuja pala + placa de empuje. Restricción de mando **Apertura**.
- Todo en **chapa de acero plegada**. Soldadura mínima, casi siempre sólo la unión del cilindro con la pala.
- **Frame**:
 - alas curvas;
 - si el frame es corto, curva en S fuera de la zona de la brida;
 - logo DEFYMOTION calado con letras esténcil sin islas.
- **Brida** según el robot (patrón ISO 9409). Valores tabulados en **ROBOT_BRIDAS**:
 - **KR140_ADECO**: PCD 160;
 - **KR_IONTEC**: 9 × M8 en Ø100, centrado Ø63.
 - Siempre avisar qué hay que confirmar (modelo exacto, pasador de posición).
- Funciones opcionales:
 - garras de pallet: ejes con garras, rodamientos UCFL y cilindro basculante con leva;
 - toma de cartón;
 - sensores inductivos con su soporte;
 - cilindros tope.

8.2 Cálculo de apriete (criterio acordado)

None

$$2 \cdot \mu \cdot F \geq m \cdot (g + a) \cdot FS \quad \rightarrow \quad F = m \cdot (g + a) \cdot FS / (2 \cdot \mu)$$

$$p = 8 \text{ bar} \cdot \mu = 0,4 \text{ (goma/cartón)} \cdot FS = 2 \cdot a = 5 \text{ m/s}^2$$

$$\text{margen de pala } 10 \text{ mm por lado} \cdot \text{apertura } 70 \text{ mm} + 10 \text{ de tolerancia} \cdot \text{luz abierta} = \text{apriete} + 45,5$$

- Se elige el cilindro ADN con su carrera. Módulos validados: **ADN-25-80** y **ADN-32-80**. Si da ADN-20, se usa el 25 y se avisa.
- Validado contra ADECO2: elige ADN-25-80, con pala de 141 mm, riel de 150 mm y chasis de 966 mm.
- **Robot chico** (por ejemplo KR 30): sumar gripper + caja y compararlo con la carga admisible **antes de construir**. Variantes para alivianar:
 - alas más cortas;

- alma y brida alivianadas;
- brida de 1/4";
- palas más bajas (2/3 superiores de la caja).
- Verificar también el centro de gravedad contra el diagrama de carga del robot.

8.3 Motor `construir_gripper.py`

- **Argumentos:**
 - `--nombre --fila --apriete --alto --masa --cajas --paso --ancho-pala;`
 - `--reusar-piezas, --rehacer <pieza>, --alas-rectas, --sin-logo;`
 - `--liviano, --alto-palas, --robot, --ala-frame.`
- **Produce:**
 - las chapas;
 - las gomas;
 - los módulos de pala móvil con restricciones;
 - el ensamble general con restricciones.
- **Verificación automática en `resultado.json`:**
 - masa real;
 - envoltente;
 - luz entre gomas abierto;
 - interferencias;
 - imagen ISO.
- **Interferencias normales que se pueden ignorar:** rosca del vástago con la tuerca; soporte con frame o con cilindro ($\approx 0,1-1 \text{ mm}^3$).

8.4 Ejemplo de referencia: KR30, 1 caja

- Caja de 300 × 300 × 300 mm y 6 kg; cilindro ADN-25-80, 393 N (FS 3,5).
- Gripper de 14,98 kg: con la caja suma unos 21 kg, contra 30 kg admisibles.
- Frame de 3/16" con alas de 110; brida de 1/4" alivianada; palas de 200 de alto.
- Envoltente de 367 × 300 × 371 mm; luz abierta de 345,5 mm.

9. Planos y DXF

9.1 Reglas

- **Un solo archivo de dibujo** con una hoja por pieza de chapa, más un **único PDF**. Todas las hojas en **A4** apaisada.
 - Salidas: `Fabricacion\Planos\<ens>_CHAPAS.dwg/.pdf, ..._CONJUNTO y Corte_laser\*.dxf.`

- **Sólo chapas propias.** Se excluyen Centro de Contenido, comerciales y gomas o plásticos.
- **Escala:** la normalizada más grande que entre (2:1, 1:1, 1:2, 1:5, 1:10, 1:20, ...).
- **Vistas:**
 - FRENTE arriba a la izquierda, LATERAL a su derecha y PLANTA debajo, alineadas;
 - DESARROLLO, ISOMÉTRICA (más chica) y notas en el primer hueco libre;
 - para ganar escala se omite PLANTA y después ISO;
 - cada vista reserva su rótulo y sus cotas, y el cajetín cuenta como zona ocupada.
- **Cotas:**
 - totales en frente y lateral;
 - en el desarrollo: totales y una cota por pliegue desde el borde;
 - enteros sin decimales;
 - **el radio de plegado no se acota** (va en las notas).
- **Notas (5 líneas):**
 1. **PIEZA (xCant por gripper);**
 2. material y espesor;
 3. desarrollo;
 4. plegados y escala;
 5. **CORTE DXF 1:1 - ISO 2768-m - SIN REBABAS.**
- **Cajetín:** título "- CHAPAS" (30 caracteres como máximo); código = nombre del ensamble (18 caracteres como máximo).
- **Criterio:** regenerar sólo si cambia el modelo. Los ajustes finos con `retocar_plano.py`, que conserva lo manual.

9.2 Lecciones de la API de dibujos

- Plantilla del usuario: `GetTemplateFile(12292)`.
- Una hoja nueva no trae borde ni cajetín: `AddBorder(def)` y `AddTitleBlock(def)` tomados de la hoja 1.
- **Vista de desarrollo:** `AddBaseView(doc, pt, escala, kDefault, estilo, ArgNotFound, ArgNotFound, nvm)` con `nvm["SheetMetalFoldedModel"] = False`. Si los opcionales van como "", dibuja la pieza plegada.
- Crear las vistas **ya a su escala**: crearlas a 1:1 y achicarlas falla en piezas grandes.
- Mover una vista con escala propia: después de `Position`, volver a fijar `Scale` y `ScaleFromBase = False`.
- **Isométrica:** vista base ISO (10759). Una proyectada en diagonal no siempre da ISO.
- **Cotas automáticas:**
 1. `view.ModelToSheetSpace(p)` da dónde cae el punto en la hoja;
 2. buscar en `DrawingCurves` la línea en esa coordenada (tolerancia de unos 0,006 cm);
 3. `GeneralDimensions.AddLinear(pt, intent1, intent2);`

4. **verificar** `dim.ModelValue × 10` contra el valor esperado; si no coincide, borrar la cota y registrar el fallo.
 - Centro de un agujero: `sheet.CreateGeometryIntent(círculo)` **sin** intención. `PointIntentEnum` falla en 2025.
 - Los círculos no tienen `StartPoint`: filtrarlos.
 - Los pliegues cortados por una escotadura son tramos colineales: agruparlos por coordenada.
 - Llamadas de soldadura: `LeaderNotes.Add(colección_2_puntos, "FILETE a3<Br/>TODO ALREDEDOR")`.
 - **Rótulos:** `ShowLabel = False` y una nota propia en negrita donde no pise.
 - **PDF:**
 - todas las hojas, en negro;
 - si el PDF está abierto en un visor, guardar con otro nombre;
 - **rasterizarlo y mirarlo** antes de entregar.
-

10. Listas de materiales

- Se arman del lado de la IA con `openpyxl`, a partir del `resultado.json` del constructor.
 - Hojas:
 - Resumen;
 - Piezas (masa unitaria × cantidad);
 - Corte de caños (barras de 6 m);
 - Herrajes;
 - Bulonería.
 - **Con fórmulas de Excel**, no valores pegados, para que el usuario pueda editar.
-

11. Animaciones: arquitectura

11.1 Por qué cuadro a cuadro

- En Inventor 2025 la API **no permite crear escenas ni acciones de presentaciones** (`.ipn`): sólo leerlas. Studio tampoco se automatiza bien.
- El modo realista es tipo boceto, y el trazado de rayos tarda minutos por cuadro.
- **Método que funciona:**
 1. mover por API;
 2. `doc.Update()`;
 3. `ActiveView.SaveAsBitmap(png, ancho, alto)`;

4. repetir;
5. armar el MP4 con ffmpeg.

11.2 Dos scripts y un JSON

Pieza	Qué hace	Cuándo se corre
<code>armar_<proyecto>.py</code>	Crea la escena copia : hace <code>SaveAs</code> de layout, gripper y subensambles móviles; suprime restricciones; libera piezas; agrega eslabones del robot y cajas extra; oculta lo duplicado. Escribe <code>escena.json</code>	Una vez (y con flags incrementales: <code>--centrador</code> , <code>--vastagos</code>)
<code>escena.json</code>	Matrices base: robot, eslabones, gripper, piezas libres; cajas del gripper; pallets; centrador; vástagos; restricciones suprimidas	Lo lee el simulador
<code>simular_<proyecto>.py</code>	Abre la escena, calcula cinemática y planes, anima y graba cuadros, y al final restituye todo	Cada render

Flags incrementales: cuando aparece algo nuevo para mover (por ejemplo los vástagos), se agrega un flag al armador que **completa** la escena existente sin rehacerla.

11.3 El simulador es determinístico

- Mismos argumentos, mismos cuadros: la cantidad de cuadros de cada movimiento depende sólo de los parámetros.
- Eso permite re-renderizar **un tramo** con los mismos números de cuadro y empalmarlo (sección 14.4).
- **Marcas**: el simulador anota en `MARCAS` el número de cuadro donde empieza y termina cada parte (`seg_ini`, `seg_fin`) y las guarda en `resultado.json`.

11.4 Esqueleto del simulador (orden de las cosas)

1. Argumentos (fases, pisos, cámara, velocidad, resolución, ángulos, flags de prueba y de tramo).

2. Abrir la escena (copia), mapear ocurrencias y **ocultar el robot armado del layout** (si no, aparecen dos robots).
 3. Cinemática (sección 12.3) y geometría de la tarea: poses de toma y deja, IK, errores de IK registrados.
 4. Planificación: patrón, orden y giro de cada caja (sección 12.7).
 5. Si `--sin-animar`: escribir `resultado.json` y salir. Así se revisa el plan sin render.
 6. Configurar la vista: planos ocultos, luz, sombras, cámara y **renders de calentamiento**.
 7. Preparar los mecanismos: garras, palas, pistones, cinta.
 8. Línea de tiempo, dentro de `try:`, con los helpers de movimiento (sección 12.8).
 9. `except Fin`: corte normal (máximo de cuadros o fin del tramo). `except Exception`: guardar la traza.
 10. `finally`: restituir garras, palas, pistones, pallet, gripper y `doc.Update()`.
 11. Comprimir los cuadros en zip y escribir `resultado.json` (marcas, cuadros, tiempo, avisos).
-

12. Animaciones: cómo se mueve cada cosa

12.1 Ocurrencias del ensamble principal

`occ.Transformation = mmake(M)` es directo y se ve después de `doc.Update()`.

12.2 Piezas dentro de subensambles

- **Subensamble rígido** (lo normal):
 1. trabajar en la **copia** del subensamble;
 2. suprimir sus restricciones;
 3. poner la `Transformation` del componente **en su documento**: coordenadas de ese `.iam`, con la matriz original leída por el armador;
 4. actualizar la cadena **de adentro hacia afuera**:

Python

```
doc_cilindro.Update(); doc_tope_pistones.Update(); doc_centrador.Update() # y el top en cuadro()
```

Si falta un `Update` intermedio, la pieza "no se mueve".

- **Subensamble flexible**: mover el **proxy** en el contexto del padre (sección 7.5).
- **Comercial con vástago** (Festo ADN):
 1. copiar el `.iam` del cilindro y el del ensamble que lo contiene (`ADN_40_50_sim.iam`, `Ensamblaje1_sim.iam`);

2. suprimir las restricciones vástago ↔ camisa (CYLINDER_XY/Z/YZ <-> PISTON_*);
 3. mover el vástago (... ADN-50-P-I_(KS)) con la misma carrera que la pieza que empuja;
 4. como todos los cilindros usan el mismo documento, se mueven juntos.
- **Comprobar midiendo:** RangeBox de la pieza (desde el proxy del top) antes y después. Si no cambió, falta un Update o el padre es flexible.

12.3 Robot KUKA (KR 140 R3200-2 PA) — cinemática

- El modelo son 5 sólidos (V01 base, V02 carrusel, V03 brazo, V04 antebrazo, V05 muñeca/brida), sin uniones. Se agregan a la escena como ocurrencias sueltas y el robot armado se oculta.
- **Ejes**, sacados de la geometría (coordenadas del ensamble del robot, Y arriba):
 - A1: eje Y por el origen;
 - A2: eje Z por (330, 645);
 - A3: eje Z por (338,02, 1994,98);
 - muñeca W: eje Z por (1291,82, 1179,32);
 - A6: gira el gripper sobre el eje vertical de la brida.
- **Brida siempre horizontal mirando abajo** (paralelogramo): a V05 se le aplica $\varphi_0 - q_2 - q_3$, con $\varphi_0 = -90 - \text{atan2}(-0,6889; 0,7249)$.
- **Directa** (celda_kin.poses): eslabón = $T_{\text{robot}} \cdot R_1(q_1) \cdot R_2(q_2 @A2) \cdot R_3(q_3 @A3) \cdot R_5(\varphi_0 - q_2 - q_3 @W) \cdot M_{\text{original}}$.
- **Inversa** (ik(G)):
 1. brida = $G \cdot BRIDA_G$;
 2. $q_1 = \text{atan2}$ en planta;
 3. muñeca = brida - offset fijo;
 4. q_2 y q_3 por el triángulo de dos eslabones, con el codo como en el modelo;
 5. q_6 por la dirección del eje X del gripper.
- **Verificar con la directa** y registrar el error: 4,5 mm en NewPack2; menos de 5–7 mm es aceptable.
- **Montaje del gripper:**
 - matriz R_G (en NewPack2: $X_g \rightarrow Z_r$, $Y_g \rightarrow Y_r$ arriba, $Z_g \rightarrow -X_r$);
 - punto $BRIDA_G$ = centro de la cara superior de la brida del gripper. **Ojo:** si la brida está oculta en el modelo, igual hay que usarla como referencia.
- **Ubicación del robot:** la matriz del robot en el layout del cliente. Si la base trae otro pie, avisar la diferencia (en ADECO fueron 345 mm).
- Para otro robot: relevar los ejes de la misma forma (cilindros grandes coaxiales de cada eslabón) y armar su celda_kin.

12.4 Objetos llevados por el gripper

- Al tomar: $M_{\text{rel}} = \text{inv}(G_{\text{actual}}) \cdot M_{\text{objeto}}$ (con la pose **real** del gripper, la de la directa).

- Mientras se lleva, en cada cuadro: $M_{\text{objeto}} = G_{\text{actual}} \cdot M_{\text{rel}}$.
- Al soltar: el objeto queda donde está. **Leer su matriz real** (`mread`) para lo que sigue: la IK tiene error y la posición no es la teórica.
- **Movimientos que terminan en contacto** (cadenas contra tope, pistones contra pallet):
 1. medir el `RangeBox` del objeto en ese momento;
 2. calcular el recorrido para que la cara toque **exacto** el tope ($dx = X_{\text{TOPE}} - \text{caja.max.x}$);
 3. el empuje empieza cuando la cara del empujador alcanza la cara del objeto: $\Delta = \max(0, Y_{\text{chapa}}(d) - y_{\text{cara}})$.

Así no quedan los 2–4 mm de superposición que deja la IK.

12.5 Garras de pallet simétricas

- Las garras giran sobre ejes con rodamientos. Para que salgan **simétricas**:
 1. se calcula sólo la garra de referencia (`EJE_REF`, la del lado $-Z_g$);
 2. la otra es la misma girada 180° alrededor del eje vertical del gripper:

Python

```
R180 = [[-1,0,0, 2*XC], [0,1,0,0], [0,0,-1,0], [0,0,0,1]]      # XC = centro del
gripper en X_g
M_ref = Rx(ang) alrededor de su eje (ly, lz) · M0_ref
M_otra = R180 · M_ref
```

- **Ángulos calibrados con imágenes de prueba** (`--test-garras`, `--test-agarre`):
 - abierta -28° : listas para bajar;
 - cerrada -7° : abrazan los extremos del pallet desde afuera;
 - plegada $+90^\circ$: hacia adentro, después de soltar el pallet, para palletizar.
- **Altura de enganche**: las palas quedan 51 mm sobre la tabla del pallet (`--luz-palas-pallet 51`), como en el modelo.
- **Secuencia**:
 1. abrir;
 2. bajar;

3. cerrar;
 4. levantar por encima de los topes del almacén (+120 mm);
 5. trasladar;
 6. bajar;
 7. abrir;
 8. subir;
 9. **plegar**.
- **Antes de renderizar, mostrar al usuario 3 vistas** (costado, abajo, ISO) de la garra abierta, cerrada y plegada. Las fotos del usuario del mecanismo real resuelven dudas.

12.6 Palas del gripper de cajas

- **Pala móvil + ensamble de empuje:** se mueven juntos por matriz. $f = 0$ abierta, como en el modelo; $f = 1$ cerrada **hasta el contacto con la caja**, no hasta el fin de carrera.
- **Cierre** = posición abierta – (cara de la pala fija + ancho apretado + 2 mm).
- **La pala móvil abre hacia el lado donde no hay caja** (sección 12.7). La pala fija puede ir contra lo apilado.
- **Primer pick con zoom (VISTA_TOMA)**, para que se vea el sistema de agarre:
 1. acercar la cámara;
 2. esperar;
 3. bajar;
 4. cerrar despacio;
 5. levantar;
 6. alejar la cámara.

12.7 Patrón de palletizado y plan de cada piso

- **Patrón:** de la imagen o las medidas del usuario, en coordenadas del pallet (desde una esquina).
 - Ejemplo NewPack2: molinete de 4 paquetes de 430 × 550 × 165 por piso.
 - **Pisos alternos espejados** ($x \rightarrow 1000 - x$) para trabar.
- **Marco canónico** del paquete: X_p = lado apretado (430), Y_p arriba, Z_p = el otro lado (550).
- **Toma:** en la cinta de entrada, contra el tope, donde está la caja del modelo.
- **Plan del piso (plan_piso):** se prueban **todas las permutaciones** del orden × giro 0° / 180° de cada caja.
 - Para cada opción se calcula la franja que ocupa la **pala móvil** (160 mm hacia su lado) y la de la **pala fija** (60 mm).
 - **Costo:** pala móvil sobre una caja ya puesta = 10; pala fija sobre caja = 1; desempate: menor giro de A6.
 - Con 4 cajas por piso son 24 × 16 combinaciones: instantáneo.
 - Resultado esperado: costo 0 en todos los pisos (se registra en **resultado.json**).

- **Llegada de cajas por la cinta:** la caja aparece 1400 mm antes y avanza hasta el tope mientras el robot trabaja (**cinta_paso** en cada cuadro).
- Las cajas que todavía no entraron quedan **ocultas pero ubicadas en la cinta**, no bajo el piso: si se esconden lejos, el plano de sombra cambia de tamaño y las sombras aparecen y desaparecen.

12.8 Línea de tiempo: helpers

```
Python
def cuadro():
    # el ÚNICO lugar que graba
    global n
    if x.max_cuadros and n >= x.max_cuadros: raise Fin()
    for ini, cant, f in TWEENS:
        # animaciones en paralelo (p. ej.
        # pistones mientras el robot viaja)
        if ini <= n < ini + cant: f((n - ini + 1) / cant)
    if not GRABAR["on"]: n += 1; return
    # modo tramo/diag: avanza el contador
    # sin grabar
    doc.Update()
    cam_set(cam_estado["v"])
    # re-aplicar la cámara SIEMPRE (si no,
    # se escapa → cuadros grises)
    app.ActiveView.SaveAsBitmap(str(FR / f"c{n:04d}.png"), x.ancho, x.alto)
    n += 1

def pasos(k): return max(1, round(k * x.velocidad))
# --velocidad escala todo

def mover(q_fin, k):
    # articular, con curva coseno (arranca y frena suave)
def mover_lineal(G_fin, k):
    # recta del gripper: interpola la pose y resuelve la
    # IK en cada cuadro
def esperar(k):
    # cuadros quietos (la cinta sigue avanzando)
def garras(ang, k); def pala(f, k); def cam_mover(vista, k)
```

- **Curva coseno:** $t = (1 - \cos(\pi \cdot i/k)) / 2$.
- **Tweens:** `TWEENS.append([n, pasos(18), lambda t: set_pist(-60*t)])`
mueve los pistones en paralelo con el viaje del robot.
- **Secuencia NewPack2 (referencia):**
 1. home;
 2. toma del pallet;
 3. levanta;
 4. viaja (los pistones se retraen en paralelo);
 5. baja al centrador, desplazado (-250 mm en X, -40 mm en Y);
 6. abre y pliega garras;
 7. el robot se aparta;
 8. **zoom al centrador;**
 9. las cadenas llevan el pallet al tope del fondo;

10. los pistones avanzan (60 mm), tocan el pallet a mitad de carrera y lo empujan contra el tope lateral;
 11. vuelve la vista general;
 12. palletizado de N pisos (primer pick con zoom);
 13. home.
-

13. Animaciones: cámara, luz, calidad y video

13.1 Cámara

Python

```
def cam_set(v):                                # v = (centro_mm, alto_visible_cm,
dirección_desde_donde_se_mira)
    c, alto, dv = v
    nrm = sum(q*q for q in dv) ** .5
    cam.Target = tg.CreatePoint(c[0]/10, c[1]/10, c[2]/10)
    cam.Eye = tg.CreatePoint(c[0]/10 + dv[0]/nrm*1000, c[1]/10 + dv[1]/nrm*1000,
c[2]/10 + dv[2]/nrm*1000)
    cam.UpVector = tg.CreateUnitVector(0, 0, 1)      # Z arriba en el layout
    cam.Perspective = False                        # ortográfica: tamaños
constantes, sin deformación
    cam.SetExtents(alto * x.ancho / x.alto, alto)
    cam.Apply()
    cam_estado["v"] = v
```

- **Re-aplicar la cámara en cada cuadro.** Si no, un **Fit** automático o un clic del usuario la "escapa" y salen cuadros grises.
- **cam_mover(vista, k)** interpola centro, alto y dirección con la curva coseno: zoom suave.
- **Vistas de NewPack2** (horizontal 1920 × 1080):
 - GENERAL: centro (-300, -900, 1250), alto 430 cm, dirección (0,2, -1, 0,75);
 - CENTRADOR: centro del destino - (150, 150), z 420, alto 190 cm, dirección (0,55, -1, 0,95);
 - TOMA: sobre la toma de la cinta + 330 mm, alto 150 cm, dirección (0,75, -1, 0,55).
- Argumentos: **--centro-vista, --alto-vista, --vista, --zoom-escala.**

13.2 Versión vertical (1080 × 1920)

- **--ancho 1080 --alto 1920.**
- Con el mismo "alto visible", el ancho visible se reduce al 56 %. Hay que:
 - **subir el alto visible** (vista general de ≈900–1000 cm en vez de 430);

- escalar los zooms (`--zoom-escala ≈ 1,6–1,8`) para que entren el centrador o la pala;
- **elegir una dirección de cámara donde la celda quede "alta"**: mirar más desde arriba (componente z mayor) o rotar para que el eje largo del layout quede vertical en pantalla.
- **Primero fotos fijas de encuadre** con `--prueba-vistas="cx,cy,cz,alto,dx,dy,dz;..."`: varias candidatas en una sola corrida de unos 20 s. Elegir con el usuario.
- **Alternativa barata**: recortar el video horizontal a 9:16. Pierde resolución (608 × 1080) y parte de la escena, así que sólo sirve para zooms.

13.3 Luz, sombras y limpieza

Python

```
for wp in acd.WorkPlanes: wp.Visible = False
ov = doc.ObjectVisibility
for p in ("AllWorkFeatures", "UserWorkPlanes", "OriginWorkPlanes", "Sketches",
"Sketches3D"): setattr(ov, p, False)
for p in ("ShowAmbientShadows", "ShowObjectShadows", "ShowGroundShadows"):
setattr(vw, p, True)
vw.ShowGroundPlane = False
doc.ActiveLightingStyle = doc.LightingStyles.Item("Luz suave")
for _ in range(3):                                # calentamiento: que las sombras estén
desde el cuadro 0
    doc.Update(); app.ActiveView.SaveAsBitmap(str(AN / "calentamiento.png"), 640,
360)
```

- **Sombras desde el primer cuadro**: renders de calentamiento antes del cuadro 0, y nada escondido "bajo el piso" (sección 12.7).
- **Ocultar duplicados**: robot armado, cajas del modelo de otro tamaño, rejas o paredes que tapen. Sólo durante el render; no se guarda.
- Imágenes de prueba de 1600 × 900; video de 1920 × 1080 (o 1080 × 1920).

13.4 Tiempos de referencia (PC del usuario, Full HD con sombras)

Qué	Cuadros	Tiempo
Video completo NewPack2, 2 pisos (velocidad 1,5)	1366	≈ 29 min
Tramo del centrador	243	≈ 5 min
Modo diagnóstico (sin grabar)	—	≈ 2 min

Qué	Cuadros	Tiempo
Prueba de 4 encuadres	4 fotos	≈ 20 s

Peso: unos 0,8 MB por PNG; 1366 cuadros suman ≈ 1,1 GB, o sea 4 zips con `partir_zip.py`.

13.5 Del lado de la IA: armar el video

1. Traer los zips (≤ 400 MB cada uno) y descomprimirlos en una carpeta de cuadros.
2. Revisar una **hoja de contacto** (mosaico de cuadros cada N) para ver la secuencia completa, y cuadros puntuales ampliados en los momentos críticos (contactos, zooms, empalmes).
3. Codificar:

Shell

```
ffmpeg -y -framerate 30 -i c%04d.png -c:v libx264 -crf 18 -preset slow -pix_fmt yuv420p salida.mp4
```

4. Enviar el MP4 y guardarlo en la PC en `Generados\<PROYECTO>\Animacion\`.

Opción de generar el video mientras se renderiza (propuesta, no implementada):

- instalar `imageio-ffmpeg` en el `.venv` de la PC;
- el simulador escribe el MP4 cuadro por cuadro con un `writer` de imageio.

Ventajas: no hay que mover gigas de PNG y el video está listo al terminar. Desventaja: para empalmar tramos hacen falta los PNG; conviene guardar las dos cosas.

14. Animaciones: proceso y retoques

14.1 Orden de trabajo para una simulación nueva

1. **Relevar** el layout (sección 5). Identificar robot, gripper, almacén, centrador, cintas, pallet y cajas, con sus topes y caras.
2. **Armar la escena copia** (`armar_*.py`) y guardar `escena.json`.
3. **Fotos del mecanismo** (garras, palas) en 3 vistas. Confirmar con el usuario.
4. `--sin-animar`: plan, IK y errores en `resultado.json`.
5. **Fotos de encuadre** (`--prueba-vistas`). Confirmar con el usuario.

6. **1 cuadro** (`--max-cuadros 1`): luz, sombras, sin duplicados.
7. **Render completo.**
8. Hoja de contacto y cuadros clave. **Mirarlos.** Después, MP4 y entrega.
9. Correcciones: tramo, no todo (14.4).

14.2 Modo diagnóstico (`--diag`)

- No graba cuadros: los helpers avanzan el contador igual, para que la secuencia sea idéntica.
- En cada etapa clave guarda:
 - `RangeBox` de las piezas en juego (pallet, chapa, vástagos, topes);
 - la etapa, el cuadro y el valor del actuador;
 - 3 fotos de cerca (ángulo del usuario, costado, planta).
- Escribe `diag.json` **después de cada etapa**: si Inventor se cae, queda lo medido.
- Con eso se encontró que la chapa de los pistones **nunca se movía** (subensamble flexible) y que el pallet quedaba 1,8 mm metido por el error de la IK.

14.3 No usar `AnalyzeInterference` en medio de la simulación

- Se llamó `acd.AnalyzeInterference(pallet, centrador)` con la escena animada y **Inventor se cerró** ("servidor RPC no disponible").
- Para controlar superposiciones durante una animación: **comparar `RangeBox`** de las caras involucradas y mirar fotos de cerca.
- `AnalyzeInterference` se usa sólo en ensambles estáticos recién construidos (verificación de grippers).
- Si Inventor se cae: avisar enseguida al usuario, pedirle que lo reabra (sin abrir ensambles) y repetir sin lo que lo tiró.

14.4 Re-renderizar sólo un tramo y empalmarlo

El usuario pidió "reutilizar el video que ya hicimos, no hacer todo desde cero":

1. El tramo se delimita con marcas: `MARCAS["seg_ini"] = n` y `MARCAS["seg_fin"] = n`.
2. `--solo-centrador` (en general `--solo-<tramo>`):
 - `GRABAR["on"] = False` al empezar;
 - se activa en `seg_ini`;
 - en `seg_fin` se lanza `raise Fin()`.
3. Como el simulador es determinístico, los cuadros nuevos salen **con los mismos números** (c0090...c0332).
4. Del lado de la IA: copiar los cuadros nuevos sobre los viejos con el mismo nombre (`cp seg/c0*.png npg/`), revisar los bordes del empalme (89/90 y 332/333) y volver a codificar.

5. Condición: **la cantidad de cuadros del tramo no cambia**. Si cambia (más o menos pasos), hay que re-renderizar desde ese punto hasta el final.

Cambios de 2–4 mm en la posición final de un objeto no se notan en los cuadros siguientes que no se re-renderizan.

14.5 Restituir al final

En el **finally**, volver a poner:

- garras y palas;
- pistones;
- pallet;
- gripper;
- `doc.Update()`;
- `SilentOperation = False`.

Las copias de escena quedan limpias para la próxima corrida. Si algo quedó en un estado raro en memoria (por ejemplo, se apagó **Flexible** en una prueba), **cerrar la escena sin guardar** (`Close(True)` + `CloseAll(True)`) y reabirla.

15. Listas de control

15.1 Proyecto nuevo de diseño (gripper, estructura)

- ☐ Datos de entrada completos (sección 0) y supuestos anotados.
- ☐ Referencias relevadas en sólo lectura → `Analisis\<PROY>\*.json` + imágenes.
- ☐ Cálculo de apriete, carga del robot y centro de gravedad. Variante liviana si hace falta.
- ☐ Construcción por script (`--nombre <PROY>`) en `Generados\<PROY>\`.
- ☐ Verificación: masa real (materiales asignados), envolvente, luz abierta, interferencias, imagen ISO **mirada**.
- ☐ Imagen al usuario. Correcciones con `--reusar-piezas` / `--rehacer`.
- ☐ Planos A4 (chapas + conjunto), PDF **mirado**, DXF por pieza.
- ☐ BOM y bulonería en xlsx con fórmulas.
- ☐ Ensamble autocontenido + zip si se envía.
- ☐ Contexto del proyecto actualizado (qué se hizo, dónde, pendientes, último pedido).

15.2 Simulación nueva

- ☐ Layout relevado: topes, caras, alturas, orígenes de pallet y cajas, subensambles **flexibles**.
- ☐ Escena copia con **SaveAs** en orden (sub primero) y restricciones de lo móvil suprimidas.
- ☐ Robot armado oculto y eslabones sueltos agregados.
- ☐ Cinemática verificada (error de IK registrado).
- ☐ Mecanismos probados con fotos: garras simétricas (abierta / cerrada / plegada), palas, pistones con vástagos.
- ☐ Plan de pisos con costo 0 (pala móvil hacia el lado libre).
- ☐ Encuadre aprobado (horizontal o vertical).
- ☐ 1 cuadro: luz suave, sombras desde el inicio, sin planos, sin duplicados.
- ☐ Render completo. Hoja de contacto y cuadros críticos mirados (contactos sin superposición, zooms, sin grises).
- ☐ MP4 entregado y copiado a la PC.

15.3 Antes de lanzar cada pedido

- ☐ Script copiado → traído → **diff** sin diferencias.
 - ☐ Id nuevo (pNNN+1).
 - ☐ **timeout_s** acorde (render completo: 3600).
 - ☐ Ningún **.json** de datos en **_pedidos**.
-

16. Plantillas de código

16.1 Vigilante mínimo (PC)

Python

```
import json, subprocess, sys, time
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]; SCRIPTS = ROOT / "scripts"
JOBS = Path.home() / "Documents" / "InventorIA" / "_pedidos";
JOBS.mkdir(parents=True, exist_ok=True)
PY = ROOT / ".venv" / "Scripts" / "python.exe"

def run(f):
    rid = f.stem; t0 = time.time()
    try:
```

```

        job = json.loads(f.read_text(encoding="utf-8"))
        target = (SCRIPTS / Path(job["script"]).name).resolve()
        assert target.parent == SCRIPTS.resolve() and target.suffix == ".py" and
target.is_file()
        p = subprocess.run([str(PY), str(target), *map(str, job.get("args", []))],
cwd=str(ROOT),
                                capture_output=True, text=True, encoding="utf-8",
errors="replace",
                                timeout=int(job.get("timeout_s", 1800)),
creationflags=0x00004000)
        res = {"codigo_salida": p.returncode, "salida": p.stdout[-60000:],
"errores": p.stderr[-20000:]}
        except subprocess.TimeoutExpired:
            res = {"codigo_salida": -2, "errores": "Tiempo agotado"}
        except Exception as e:
            res = {"codigo_salida": -1, "errores": repr(e)}
        res.update(id=rid, segundos=round(time.time() - t0, 1))
        (JOBS / f"{rid}.resultado.json").write_text(json.dumps(res,
ensure_ascii=False, indent=1), encoding="utf-8")
        f.replace(f.with_suffix(".hecho"))

while True:
    for f in sorted(JOBS.glob("*.json")):
        if not f.name.endswith(".resultado.json"):
            run(f)
        (JOBS / "vigilante.activo").write_text(time.strftime("%Y-%m-%dT%H:%M:%S"),
encoding="utf-8")
        time.sleep(2)

```

vigilante.bat: cd /d %~dp0.. && .venv\Scripts\python.exe
scripts\vigilante.py & pause

16.2 Encabezado de todo script de la PC

Python

```

"""<Qué hace> (sobre COPIA; no guarda originales).
Uso: python <script>.py [--args] -> Generados\\<PROY>\\resultado.json"""
import argparse, json, sys, time, traceback
from pathlib import Path
sys.stdout.reconfigure(encoding="utf-8")
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "server"))
sys.path.insert(0, str(Path(__file__).resolve().parent))
import inventor_mcp as inv          # _inventor, _safe, _items, constantes
S = inv._safe

```

```

app = inv._inventor(start_if_needed=False); app.SilentOperation = True; tg =
app.TransientGeometry
log = {"avisos": []}; T0 = time.time()
try:
    ... # trabajo
except Exception as exc:
    log["error"] = repr(exc); log["traza"] = traceback.format_exc()[-2000:]
finally:
    S(lambda: setattr(app, "SilentOperation", False))
    OUT.joinpath("resultado.json").write_text(json.dumps(log, ensure_ascii=False,
indent=1, default=str), "utf-8")
    print(json.dumps(log, ensure_ascii=False, default=str)[-4000:])

```

16.3 Mover un componente dentro de un subensamble, en la dirección del mundo

```

Python
def preparar_movil(top_occ_padre, nombre_hijo, M_padre_en_mundo):
    """Devuelve (proxy, M0, u_local, doc_padre). Funciona con padres flexibles o
    rígidos."""
    padre_doc = top_occ_padre.Definition.Document
    hijo = next(o for o in inv._items(top_occ_padre.Definition.Occurrences) if
o.Name == nombre_hijo)
    u_local = rotv(inv_rt(M_padre_en_mundo), (0, 1, 0))          # +Y del mundo en
coords del padre
    return hijo, mread(hijo.Transformation), u_local, padre_doc

def mover_d(hijo, M0, u, padre_doc, d):
    hijo.Transformation = mmake(mmul(tr(*[u[i]*d for i in range(3)]), M0))
    padre_doc.Update()          # + los docs intermedios, de adentro hacia afuera

```

(Si el hijo está dentro de una ocurrencia **flexible**, usar `occ_flexible.SubOccurrences` en el contexto de su padre; sección 7.5.)

16.4 Re-render de un tramo

```

Python
GRABAR = {"on": not x.solo_tramo}
...
MARCAS["seg_ini"] = n; GRABAR["on"] = True
... # movimientos del tramo
MARCAS["seg_fin"] = n
if x.solo_tramo: raise Fin()

```

16.5 Hoja de contacto (IA)

Python

```
from PIL import Image
fr = list(range(90, 345, 16)); ims = [Image.open(f"c{i:04d}.png").resize((320,
180)) for i in fr]
W = Image.new("RGB", (1280, 180 * ((len(ims) + 3)//4)))
for k, im in enumerate(ims): W.paste(im, ((k % 4)*320, (k//4)*180))
W.save("hoja.png")
```

17. Errores típicos y su solución

Síntoma	Causa	Solución
<code>MK_E_UNAVAILABLE / 0x80080005</code>	Inventor y Python en sesiones o permisos distintos	Misma sesión y mismo nivel; <code>IDispatch</code> dinámico
Wrappers rotos, documento sin <code>ComponentDefinition</code>	Caché <code>gen_py</code>	<code>Dispatch(unk.QueryInterface(IID_IDispatch))</code>
El script se cuelga	Diálogo de Inventor esperando	<code>SilentOperation = True</code> ; el usuario cierra el diálogo
El vigilante no responde	Render largo (no late) o diálogo	Esperar el <code>resultado.json</code> ; reabrir el vigilante
<code>FileExistsError</code> al marcar <code>.hecho</code>	<code>rename</code> sobre uno existente	<code>Path.replace()</code>
Corrió la versión vieja del script	Caché del puente o pedido antes que el script	Staging nuevo + <code>diff</code> de verificación + pedido al final
Un <code>.json</code> de datos se ejecutó	El vigilante toma todo <code>*.json</code>	Datos como <code>.txt</code>
Pieza extruida al revés	Sentido según el boceto	<code>RangeBox</code> y reextruir
Restricción deja la pieza dada vuelta	Convención de signo	Calibrar con offset 0 y medir

Síntoma	Causa	Solución
DXF sale del canto / espesor 3,05	Placa no convertida a chapa sobre la cara grande	Convertir a chapa con espesor real
Cota a un centro falla	<code>PointIntentEnum</code> en 2025	Intent del círculo sin argumento
Isométrica de canto	Proyectada	Vista base ISO 10759
Vista desarmada al moverla	Escala propia	Re-fijar <code>Scale</code> y <code>ScaleFromBase = False</code>
Masa absurda	Material "Genérico"	Asignar material real
Prueba de perturbación corrió piezas del usuario	Se hizo sobre el original abierto	Sólo en copias ; el usuario cierra sin guardar
Dos robots en el video	Robot armado del layout visible	Ocultarlo (<code>Visible = False</code>)
Cajas o bandejas dadas vuelta al dejar	Origen o ejes de la pieza	Marco canónico desde el <code>RangeBox</code>
El gripper choca al dejar	Pala móvil hacia lo apilado	<code>plan_piso</code> (pala móvil hacia el lado libre)
Garras asimétricas	Cada una calculada por separado	La segunda = $R180 \cdot la primera$
Garras se pliegan hacia afuera	Signo del ángulo	Plegada +90° (hacia adentro); calibrar con fotos
Cuadros grises al final	La cámara se escapó (Fit, clic)	Re-aplicar la cámara en cada <code>cuadro()</code>
Sin sombras al principio	Objetos ocultos lejos; sombras sin calcular	Ocultos en un lugar plausible + renders de calentamiento
Una pieza de un subensamble "no se mueve"	Falta un <code>Update()</code> de la cadena	<code>Update</code> de cada doc, de adentro hacia afuera
Sigue sin moverse (la caja no cambia)	Ocurrencia padre flexible	Mover el proxy en el contexto del padre; no apagar <code>Flexible</code>

Síntoma	Causa	Solución
Al apagar Flexible se corren otras piezas	Recalcula el subensamblé	No apagarlo; si pasó, cerrar sin guardar y reabrir
Pallet superpuesto 2–4 mm con la chapa o el tope	Error de la IK (4,5 mm)	Recorridos calculados desde el RangeBox medido
Pallet cae encima de la chapa del centrador	Chapa sin retraer	Retraer antes de soltar (tween) y verificar con --diag
Inventor se cerró ("RPC no disponible")	AnalyzeInterference en la escena animada	No usarlo en simulación; comparar RangeBox
Zip mayor de 400 MB	Full HD con sombras	partir_zip.py (zips < 350 MB)
Video vertical mal encuadrado	Mismo alto visible que el horizontal	Alto visible ×2, --zoom-escala , dirección más cenital; fotos de prueba

18. Datos de referencia

18.1 Constantes del proyecto

Dato	Valor
Presión de cálculo	8 bar
μ goma / cartón	0,4
FS apriete	2
Aceleración	5 m/s ²
Densidad acero	7,85 g/cm ³
Escalas de plano	2:1, 1:1, 1:2, 1:5, 1:10, 1:20, 1:25, 1:30, 1:40, 1:50

Dato	Valor
Video	1920 × 1080 (o 1080 × 1920), 30 fps, x264 crf 18, yuv420p
Luz	"Luz suave", sombras ambiente + objeto + suelo, sin plano de suelo

18.2 KUKA KR 140 R3200-2 PA (modelo de ADECO2)

- Ejes en el ensamble del robot (mm, Y arriba): A2 (330, 645), A3 (338,02, 1994,98), muñeca (1291,82, 1179,32).
- $\phi_0 = -90 - \text{atan2}(-0,6889; 0,7249)$. Brida PCD 160.
- Archivo: [12_ADECO2\KUKA\KUKA-KR_140_R3200_2_PA-ASM.iam](#) (eslabones V01–V05).

18.3 NewPack2 (valores medidos, como ejemplo de lo que hay que relevar)

Elemento	Valor
Centrador: cadenas	z = 360; x de -2566 a -566
Tope del fondo	x = -566,4
Tope lateral	y = -1121,2
Chapa de pistones extendida	cara en y = -2321,2; carrera de 60 mm (retraída = -2381,2)
Pallet	1000 (X) × 1200 (Y) × 152; origen = centro de la base
Pallet centrado	x -1566,4..-566,4, y -2321,2..-1121,2
Pallet al soltarlo	desplazado -250 mm en X y -40 mm en Y; queda 18 mm separado de la chapa retraída
Paquete	430 × 550 × 165, pinwheel de 4 por piso, pisos espejados
Garras	-28° abiertas / -7° cerradas / +90° plegadas; palas 51 mm sobre la tabla del pallet
Paso sobre topes del almacén	+120 mm sobre z 1504,8

Elemento	Valor
Error de IK	4,5 mm

18.4 Comerciales usados

- Festo:
 - ADN-25-80 y ADN-32-80 (palas);
 - ADN-40/50 (pistones del centrador);
 - DSBC-32-300, DSBC-50-100 (basculante de las garras).
- HIWIN: HGH15CA, HGW20.
- Rodamientos UCFL204.
- Sensores Pepperl+Fuchs NBB4.
- Neumática Festo: isla VTUG, GR-QS-6, MS4-LR, VUVG.

Mantener esta guía viva: cada lección nueva (error con su solución, valor calibrado, regla acordada con el usuario) se agrega en su sección y en la tabla 17. Lo propio de cada proyecto (qué se hizo, dónde quedó, último pedido, pendientes) va en el [InventorIA_contexto.md](#) de ese proyecto.